LAPORAN PRAKTIKUM
Modul 4: Mengoperasikan Database dan Generate Data Otomatis dengan Laravel



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Adapun tujuan yang ingin dicapai pada praktikum Modul 4 (Mengoperasikan Database dan Generate Data Otomatis dengan Laravel) melalui pengembangan aplikasi LaporBanjir berbasis database MySQL ini adalah sebagai berikut.

1. **Memahami cara menghubungkan aplikasi Laravel dengan server MySQL,** yaitu dengan membuat database baru dan mengatur parameter koneksinya pada file .env.
2. **Membuat migration tabel laporan,** sehingga struktur tabel beserta kolom dan tipe datanya terbentuk melalui kode PHP tanpa menulis perintah SQL manual.
3. **Membuat model Laporan,** yang berperan sebagai perantara antara tabel laporan dan kode program.
4. **Menyimpan data dari form pelaporan ke database** menggunakan Laporan::create() setelah data melewati proses validasi.
5. **Menampilkan kembali data laporan dari database** pada halaman daftar laporan beserta status genangan Waspada, Siaga, dan Awas.
6. **Mengisi tabel dengan data dummy memakai seeder dan Faker,** agar halaman daftar laporan dapat diuji tanpa harus mengisi form satu per satu.

2. Alat dan Bahan

2.1 Perangkat Lunak

Perangkat lunak yang digunakan selama praktikum beserta versinya adalah sebagai berikut.

Perangkat Lunak	**Versi / Keterangan**
Windows 11 Pro	Sistem operasi 64-bit
XAMPP (Apache & MySQL)	Terpasang pada D:\xampp
MySQL / MariaDB	10.4.32, database db_laporbanjir
phpMyAdmin	Untuk mengelola dan memeriksa database
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.69.3
Visual Studio Code	Editor kode
Git dan GitHub	Untuk menyimpan dan membagikan kode

3. Langkah-Langkah Praktikum

Praktikum ini melanjutkan aplikasi LaporBanjir yang dibuat pada Modul 3. Kalau pada modul sebelumnya data laporan hanya disimpan sebagai array di dalam controller sehingga hilang saat halaman ditutup, sekarang data tersebut disimpan ke database MySQL. Halaman form, halaman konfirmasi, dan halaman daftar laporan tetap dipakai, hanya sumber datanya yang berubah. Berikut tahapan pengerjaannya.

3.1 Membuat Database dan Mengatur Koneksi

Langkah pertama adalah membuat database baru bernama db_laporbanjir. Saya membuatnya melalui terminal dengan perintah berikut, walaupun cara yang sama bisa dilakukan lewat menu New di phpMyAdmin.

```
mysql -u root -e "CREATE DATABASE db_laporbanjir CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;"
```

Setelah database dibuat, saya membuka file .env pada folder utama project dan mengubah konfigurasi koneksinya. Sebelumnya project ini masih memakai SQLite, sehingga saya mengganti DB_CONNECTION menjadi mysql dan menyesuaikan nama database, host, port, serta user.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laporbanjir
DB_USERNAME=root
DB_PASSWORD=
```

DB_CONNECTION menentukan driver database yang dipakai. DB_HOST dan DB_PORT menunjuk ke server database lokal, sedangkan DB_DATABASE diarahkan ke db_laporbanjir. Karena XAMPP secara bawaan memakai user root tanpa password, DB_USERNAME diisi root dan DB_PASSWORD dibiarkan kosong. Setelah file .env diubah, saya menjalankan php artisan config:clear supaya Laravel membaca konfigurasi terbaru dan tidak memakai cache lama.

3.2 Membuat Migration Tabel Laporan

Migration dipakai untuk membentuk tabel langsung dari kode PHP, jadi saya tidak perlu menulis perintah CREATE TABLE sendiri. Saya membuat file migration baru dengan perintah php artisan make:migration create_laporan_table, lalu menyesuaikan method up() agar membuat tabel laporan.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('laporan', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelapor');
            $table->string('lokasi');
            $table->integer('tinggi_genangan');
            $table->date('tanggal_kejadian');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('laporan');
    }
};
```

Tabel laporan memiliki tujuh kolom. Kolom id sebagai primary key, nama_pelapor dan lokasi bertipe string, tinggi_genangan bertipe integer karena isinya angka dalam satuan sentimeter, tanggal_kejadian bertipe date, serta created_at dan updated_at dari timestamps(). Method down() berisi Schema::dropIfExists('laporan') supaya tabel bisa dihapus saat rollback.

3.3 Membuat Model Laporan

Model menjadi perantara antara tabel dan kode program. Saya membuat model Laporan, lalu mengisi properti $table dengan nama tabel dan $fillable dengan kolom yang boleh diisi lewat mass assignment.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Laporan extends Model
{
    /**
     * Nama tabel yang digunakan oleh model ini.
     *
     * @var string
     */
    protected $table = 'laporan';

    /**
     * Kolom yang boleh diisi secara massal (mass assignment).
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'nama_pelapor',
        'lokasi',
        'tinggi_genangan',
        'tanggal_kejadian',
    ];
}
```

Properti $table memastikan model terhubung ke tabel laporan. Properti $fillable memuat nama_pelapor, lokasi, tinggi_genangan, dan tanggal_kejadian. Tanpa $fillable, pemanggilan Laporan::create() akan ditolak Laravel dengan MassAssignmentException.

3.4 Menyesuaikan Controller agar Memakai Database

Controller yang sebelumnya menyimpan data di session saya ubah agar membaca dan menulis ke tabel laporan. Method index() sekarang memakai Laporan::all() untuk mengambil seluruh

data, method store() menyimpan data form dengan Laporan::create(), dan method konfirmasi() menerima mode Laporan melalui route model binding.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;

class LaporanController extends Controller
{
    /**
     * Halaman daftar laporan banjir.
     * Data diambil dari database menggunakan Eloquent (Laporan::all()).
     */
    public function index()
    {
        $laporans = Laporan::all();

        return view('laporan.index', compact('laporans'));
    }

    /**
     * Halaman form pelaporan banjir.
     */
    public function create()
    {
        return view('laporan.form');
    }

    /**
     * Memproses data form yang dikirim melalui method POST,
     * lalu menyimpannya ke tabel laporan menggunakan Laporan::create().
     */
    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama_pelapor'     => 'required|string|max:100',
            'lokasi'           => 'required|string|max:150',
            'tinggi_genangan'  => 'required|numeric|min:0',
            'tanggal_kejadian' => 'required|date',
        ]);

        // Menyimpan data ke database melalui Eloquent.
        $laporan = Laporan::create($validated);

        return redirect()
            ->route('laporan.konfirmasi', $laporan)
            ->with('success', 'Laporan berhasil disimpan ke database.');
    }

    /**
     * Halaman konfirmasi yang menampilkan kembali data laporan
     * yang baru saja tersimpan di database.
     */
```

Pada store(), data dari form divalidasi terlebih dahulu. Kolom nama_pelapor dan lokasi wajib diisi, tinggi_genangan harus berupa angka minimal nol, dan tanggal_kejadian harus berupa tanggal. Setelah valid, data disimpan dengan Laporan::create($validated) dan pengguna diarahkan ke halaman konfirmasi milik laporan yang baru dibuat.

3.5 Menyesuaikan Route

Route disesuaikan agar halaman konfirmasi menerima id laporan. Parameter {laporan} pada route tersebut otomatis diubah menjadi objek Laporan oleh Laravel.

```php
<?php

use App\Http\Controllers\LaporanController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('laporan.index');
});

// Halaman daftar laporan banjir.
Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');

// Halaman form pelaporan banjir.
Route::get('/laporan/create', [LaporanController::class, 'create'])->name('laporan.create');

// Memproses form pelaporan (method POST) dan menyimpan ke database.
Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');

// Halaman konfirmasi data laporan yang tersimpan.
Route::get('/laporan/{laporan}/konfirmasi', [LaporanController::class, 'konfirmasi'])->name('laporan.konfirmasi');
```

Terdapat empat route. laporan.index menampilkan daftar laporan, laporan.create menampilkan form, laporan.store memproses data yang dikirim dengan method POST, dan laporan.konfirmasi menampilkan hasil penyimpanan berdasarkan laporan yang dipilih.

3.6 Menyesuaikan Tampilan

Karena data laporan sekarang berupa model dan ada kolom tanggal_kejadian yang belum ada pada Modul 3, beberapa view perlu disesuaikan. Form pelaporan saya tambahkan input tanggal kejadian agar sesuai dengan kolom pada tabel.

```blade
@extends('layouts.app')

@section('title', 'Buat Laporan')

@section('content')
    <div class="card">
        <h2>Form Pelaporan Banjir</h2>

        <x-alert type="info" message="Isi data di bawah ini sesuai kondisi banjir di wilayah Anda." />

        <form action="{{ route('laporan.store') }}" method="POST">
            @csrf

            <label for="nama_pelapor">Nama Pelapor</label>
            <input type="text" id="nama_pelapor" name="nama_pelapor" value="{{ old('nama_pelapor') }}" placeholder="Contoh: Ahmad Fauzi">
            @error('nama_pelapor')
                <div class="error-text">{{ $message }}</div>
            @enderror

            <label for="lokasi">Lokasi Kejadian</label>
            <input type="text" id="lokasi" name="lokasi" value="{{ old('lokasi') }}" placeholder="Contoh: Kp. Cieunteung, Kec. Baleendah">
            @error('lokasi')
                <div class="error-text">{{ $message }}</div>
            @enderror

            <label for="tinggi_genangan">Tinggi Genangan Air (cm)</label>
            <input type="number" id="tinggi_genangan" name="tinggi_genangan" value="{{ old('tinggi_genangan') }}" min="0" placeholder="Contoh: 45">
            @error('tinggi_genangan')
                <div class="error-text">{{ $message }}</div>
            @enderror

            <label for="tanggal_kejadian">Tanggal Kejadian</label>
            <input type="date" id="tanggal_kejadian" name="tanggal_kejadian" value="{{ old('tanggal_kejadian', date('Y-m-d')) }}">
            @error('tanggal_kejadian')
                <div class="error-text">{{ $message }}</div>
            @enderror

            <button type="submit">Kirim Laporan</button>
        </form>
    </div>
@endsection
```

Halaman daftar laporan tetap memakai @forelse dan partial laporan-card, hanya saja data yang diulang sekarang berisi model Laporan, bukan array.

```blade
@extends('layouts.app')

@section('title', 'Daftar Laporan')

@section('content')
    <h2>Daftar Laporan Banjir</h2>

    @forelse ($laporans as $laporan)
        @include('partials.laporan-card', ['laporan' => $laporan])
    @empty
        <x-alert type="warning" message="Belum ada laporan banjir yang masuk." />
    @endforelse
@endsection
```

Partial laporan-card menentukan status genangan dari nilai tinggi_genangan. Tinggi di bawah 30 cm diberi status Waspada, 30 sampai 70 cm Siaga, dan di atas 70 cm Awas. Nilai tersebut diambil dari atribut model, misalnya $laporan->tinggi_genangan.

```blade
@php
    $tinggi = $laporan->tinggi_genangan;

    // Menentukan status genangan berdasarkan tinggi air (cm).
    if ($tinggi < 30) {
        $status = 'Waspada';
        $badge = 'badge-waspada';
    } elseif ($tinggi <= 70) {
        $status = 'Siaga';
        $badge = 'badge-siaga';
    } else {
        $status = 'Awas';
        $badge = 'badge-awas';
    }
@endphp

<div class="card">
    <h2>
        {{ $laporan->lokasi }}
        <span class="badge {{ $badge }}">{{ $status }}</span>
    </h2>
    <p><strong>Pelapor:</strong> {{ $laporan->nama_pelapor }}</p>
    <p><strong>Tinggi Genangan:</strong> {{ $tinggi }} cm</p>
    <p><strong>Tanggal Kejadian:</strong> {{ \Illuminate\Support\Carbon::parse($laporan->tanggal_kejadian)->format('d-m-Y') }}</p>
    <p><strong>Dilaporkan:</strong> {{ $laporan->created_at->format('d-m-Y H:i') }}</p>
</div>
```

Halaman konfirmasi menampilkan kembali data laporan yang tersimpan beserta komponen <x-alert>.

```blade
@extends('layouts.app')

@section('title', 'Konfirmasi Laporan')

@section('content')
    <div class="card">
        <h2>Konfirmasi Laporan</h2>

        <x-alert type="success" message="Laporan berhasil disimpan ke database. Terima kasih telah melaporkan!" />

        @php
            $tinggi = $laporan->tinggi_genangan;

            // Menentukan status genangan berdasarkan tinggi air (cm).
            if ($tinggi < 30) {
                $status = 'Waspada';
            } elseif ($tinggi <= 70) {
                $status = 'Siaga';
            } else {
                $status = 'Awas';
            }
        @endphp

        <table class="detail-table">
            <tr>
                <th>Nama Pelapor</th>
                <td>{{ $laporan->nama_pelapor }}</td>
            </tr>
            <tr>
                <th>Lokasi Kejadian</th>
                <td>{{ $laporan->lokasi }}</td>
            </tr>
            <tr>
                <th>Tinggi Genangan Air</th>
                <td>{{ $tinggi }} cm</td>
            </tr>
            <tr>
                <th>Tanggal Kejadian</th>
                <td>{{ \Illuminate\Support\Carbon::parse($laporan->tanggal_kejadian)->format('d-m-Y') }}</td>
            </tr>
            <tr>
                <th>Status Genangan</th>
                <td>{{ $status }}</td>
            </tr>
        </table>
    </div>

    <a href="{{ route('laporan.index') }}">&larr; Lihat Daftar Laporan</a>
@endsection
```

3.7 Membuat Seeder Laporan dengan Faker

Tabel laporan masih kosong setelah migration. Supaya halaman daftar bisa langsung diuji, saya membuat seeder LaporanSeeder yang mengisi data dummy memakai Faker dengan locale id_ID.

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Faker\Factory as Faker;

class LaporanSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * Menghasilkan data dummy laporan banjir menggunakan Faker
     * dengan locale id_ID.
     */
    public function run(): void
    {
        $faker = Faker::create('id_ID');

        // Daftar kecamatan di Kabupaten Bandung sebagai pilihan lokasi.
        $kecamatan = [
            'Dayeuhkolot',
            'Baleendah',
            'Bojongsoang',
            'Rancaekek',
            'Majalaya',
        ];

        for ($i = 0; $i < 20; $i++) {
            DB::table('laporan')->insert([
                'nama_pelapor'     => $faker->name,
                'lokasi'           => 'Kec. ' . $faker->randomElement($kecamatan) . ', Kab. Bandung',
                'tinggi_genangan'  => $faker->numberBetween(10, 150),
                'tanggal_kejadian' => $faker->dateTimeBetween('-1 month', 'now')->format('Y-m-d'),
                'created_at'       => now(),
                'updated_at'       => now(),
            ]);
        }
    }
}
```

Faker::create('id_ID') menghasilkan nama orang Indonesia. Lokasi dipilih acak dari lima kecamatan di Kabupaten Bandung, yaitu Dayeuhkolot, Baleendah, Bojongsoang, Rancaekek, dan Majalaya. Tinggi genangan diacak antara 10 sampai 150 cm, sedangkan tanggal kejadian diambil acak dalam satu bulan terakhir. Perulangan dijalankan 20 kali sehingga tabel terisi 20 baris data. Seeder ini kemudian didaftarkan pada DatabaseSeeder melalui $this->call([...]).

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        // User::factory()->create([
        //     'name' => 'Test User',
        //     'email' => 'test@example.com',
        // ]);

        $this->call([
            LaporanSeeder::class,
        ]);
    }
}
```

3.8 Menjalankan Migration dan Seeder

Setelah semua file siap, saya menjalankan migration dan seeder sekaligus dengan perintah php artisan migrate:fresh --seed. Perintah migrate:fresh menghapus seluruh tabel lalu membangunnya ulang dari migration, sedangkan opsi --seed menjalankan seeder yang terdaftar.

```
php artisan config:clear
php artisan migrate:fresh --seed
```

Prosesnya berjalan tanpa galat. Tabel laporan berhasil dibuat dan 20 data dummy berhasil dimasukkan ke dalamnya.

3.9 Menjalankan dan Menguji Aplikasi

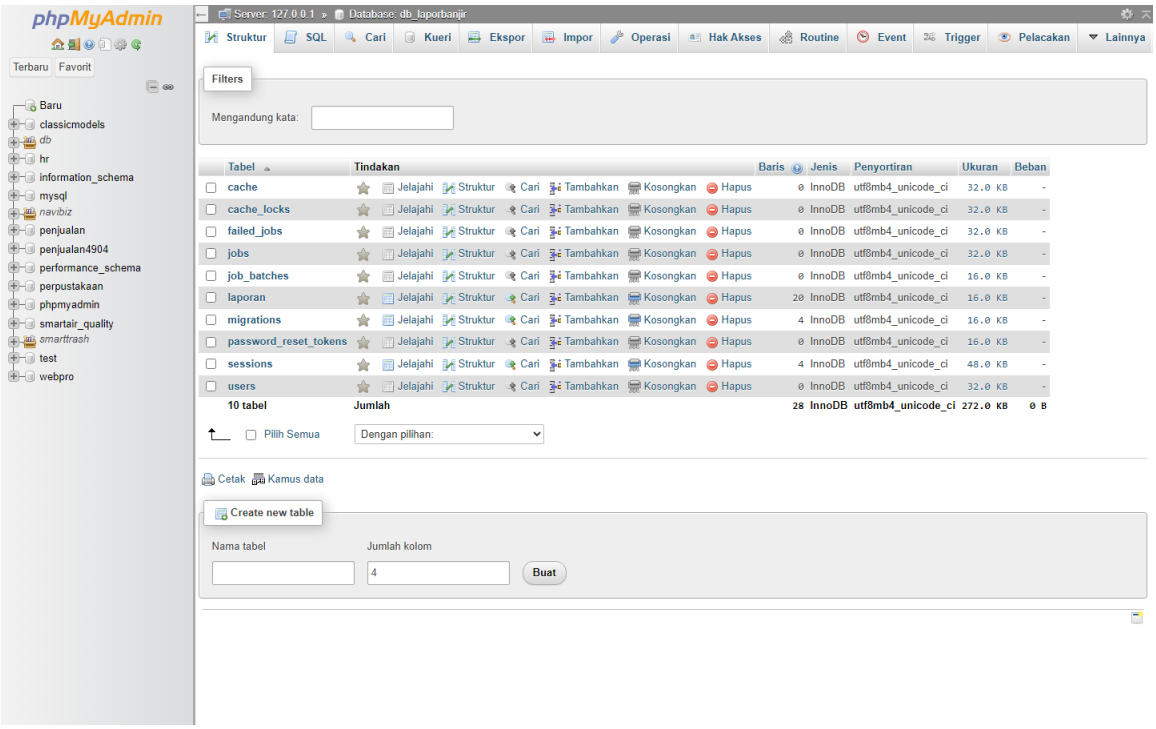
Server pengembangan dijalankan dengan php artisan serve, lalu aplikasi dibuka melalui browser.

```
php artisan serve
```

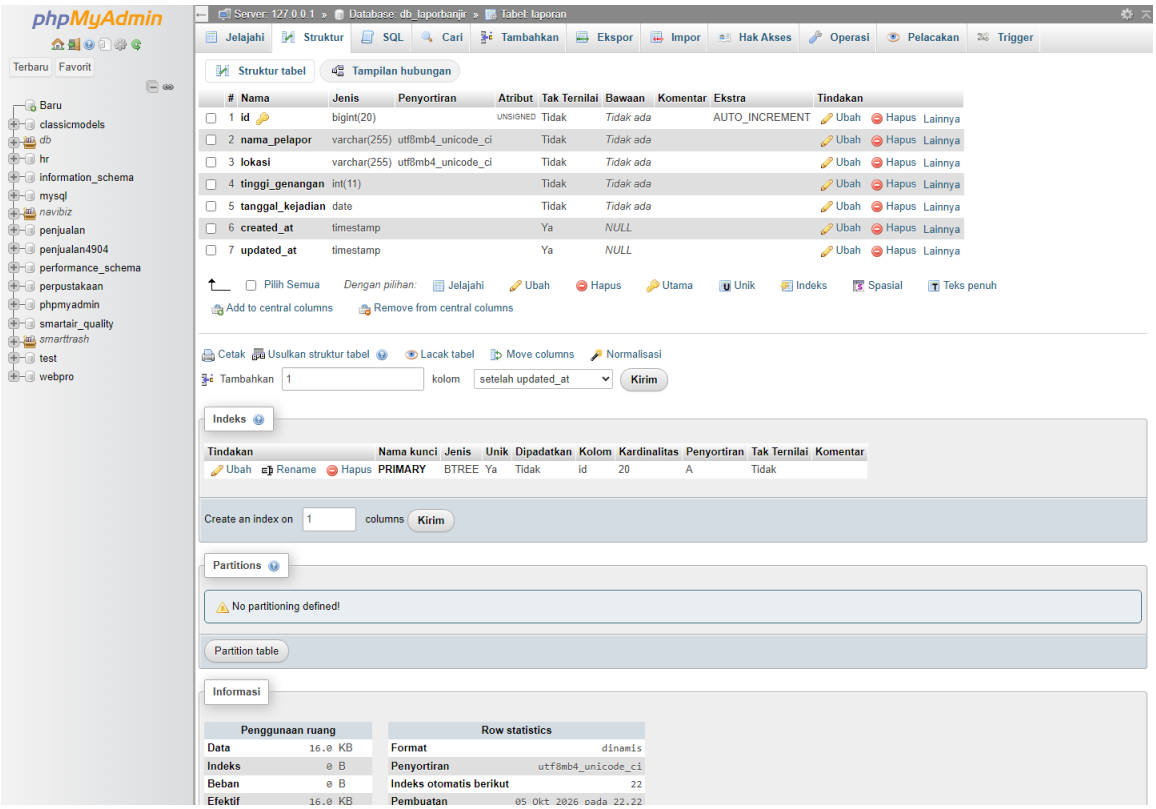
Halaman form dibuka melalui /laporan/create, halaman daftar melalui /laporan, dan halaman konfirmasi muncul setelah form dikirim. Saya mengisi form, memastikan data tersimpan ke database, lalu memeriksanya di phpMyAdmin.

4. Hasil dan Dokumentasi Program

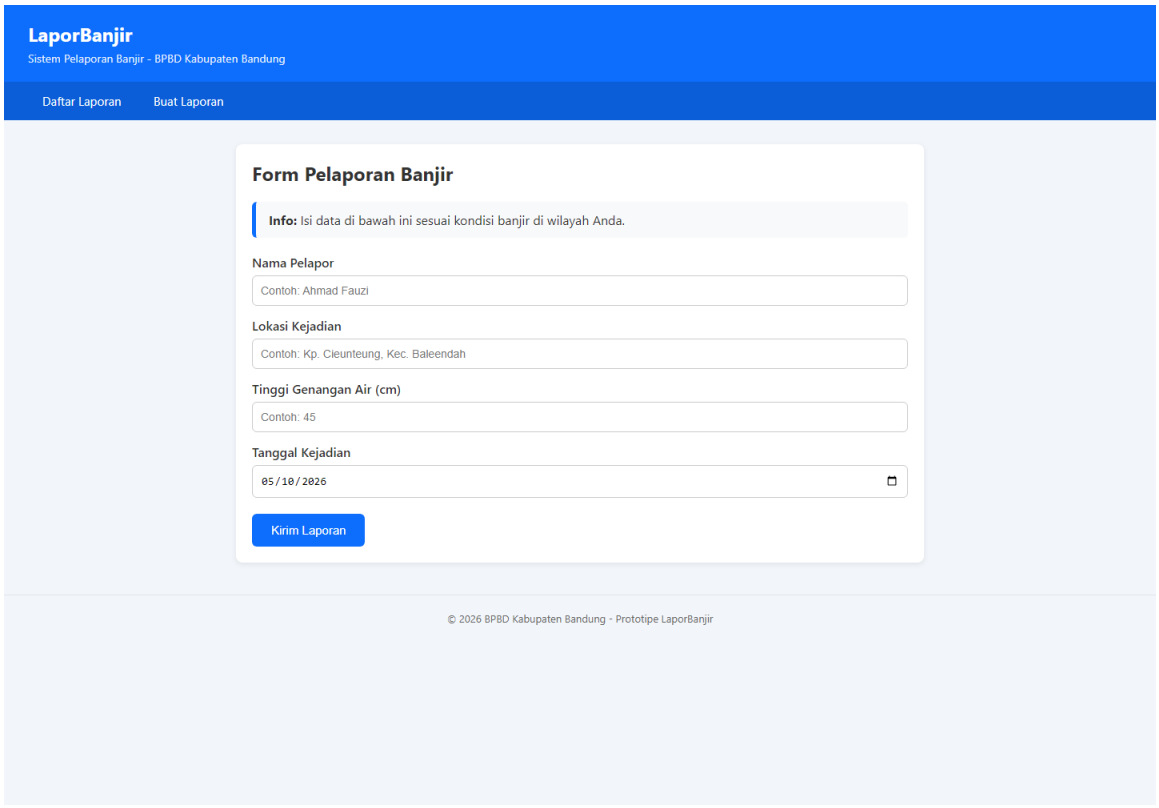
Aplikasi LaporBanjir berhasil dijalankan dengan data yang tersimpan di database MySQL. Berikut dokumentasi hasilnya.



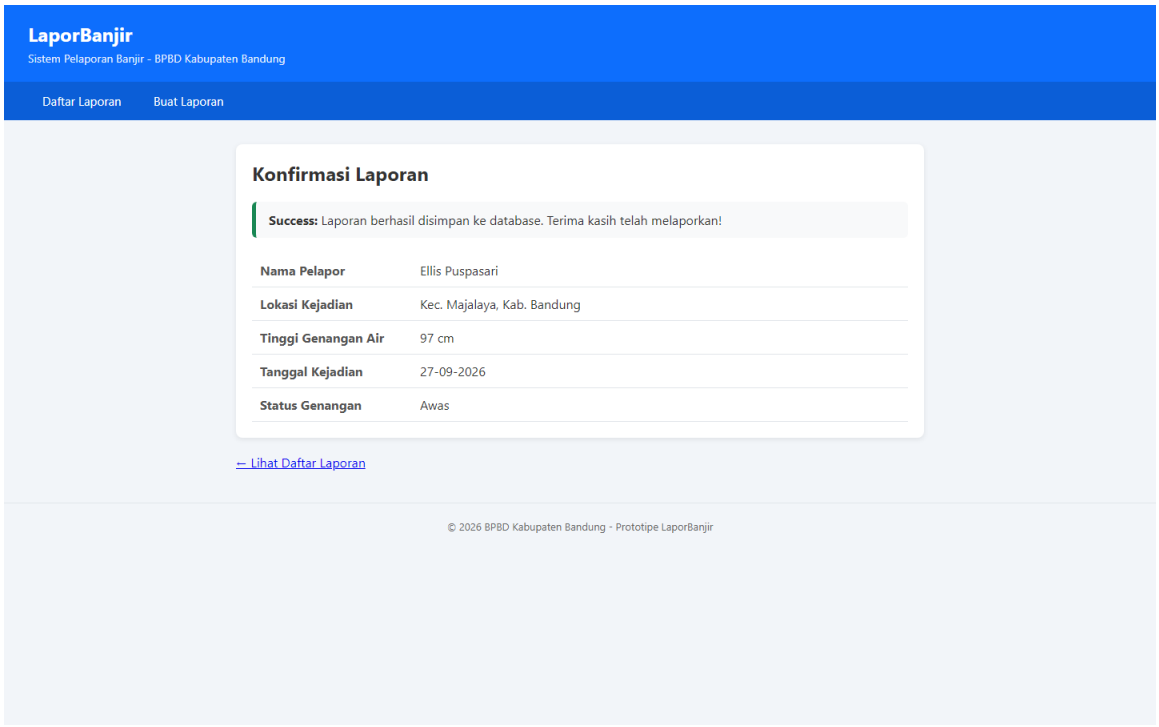
Gambar 1 menampilkan database db_laporbanjir beserta tabel yang terbentuk. Tabel laporan terlihat dengan jumlah 20 baris, sedangkan tabel lain seperti users, cache, dan sessions dibuat otomatis oleh Laravel.



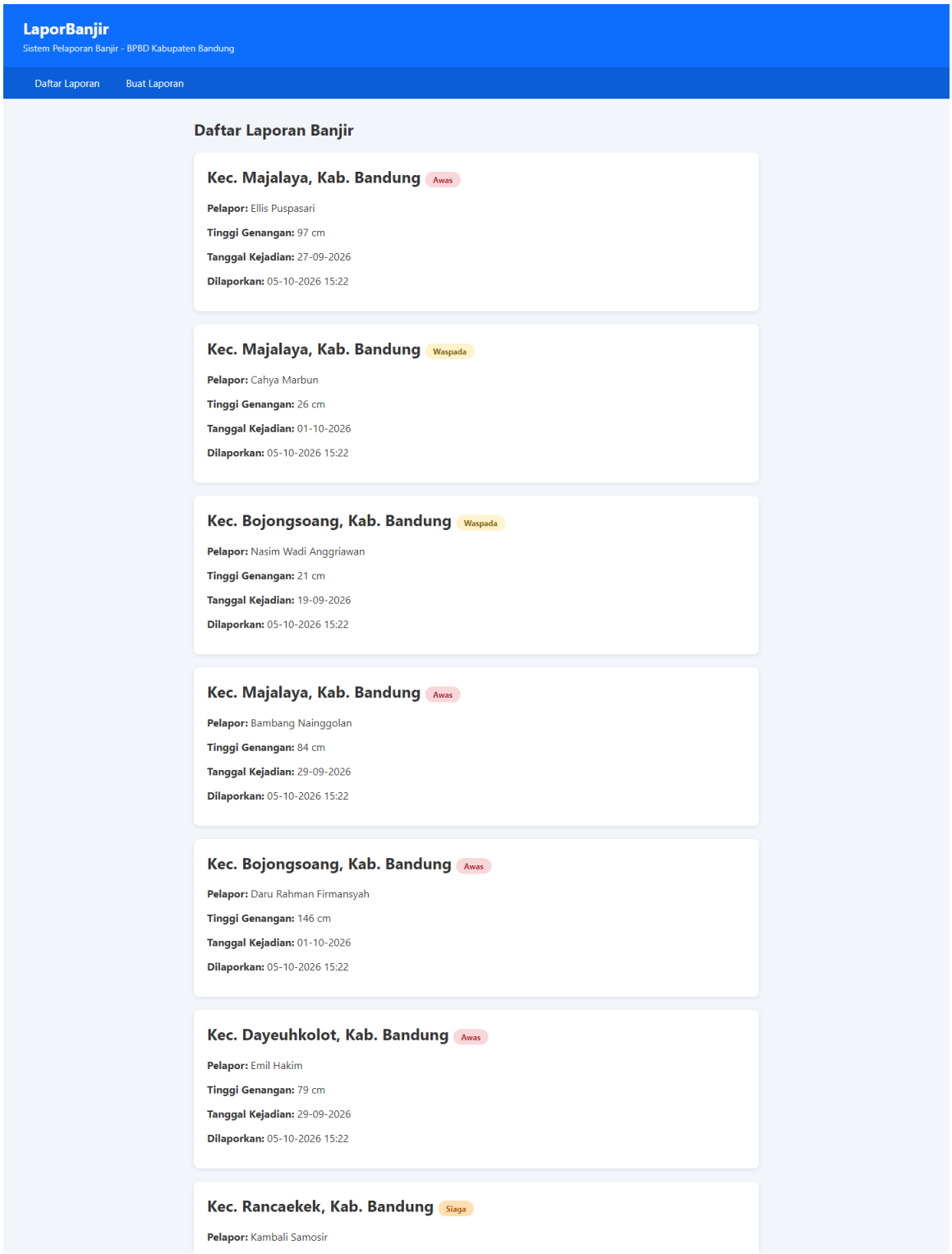
Gambar 2 menampilkan struktur tabel laporan. Terdapat kolom id sebagai primary key, nama_pelapor, lokasi, tinggi_genangan bertipe int, tanggal_kejadian bertipe date, serta created_at dan updated_at.



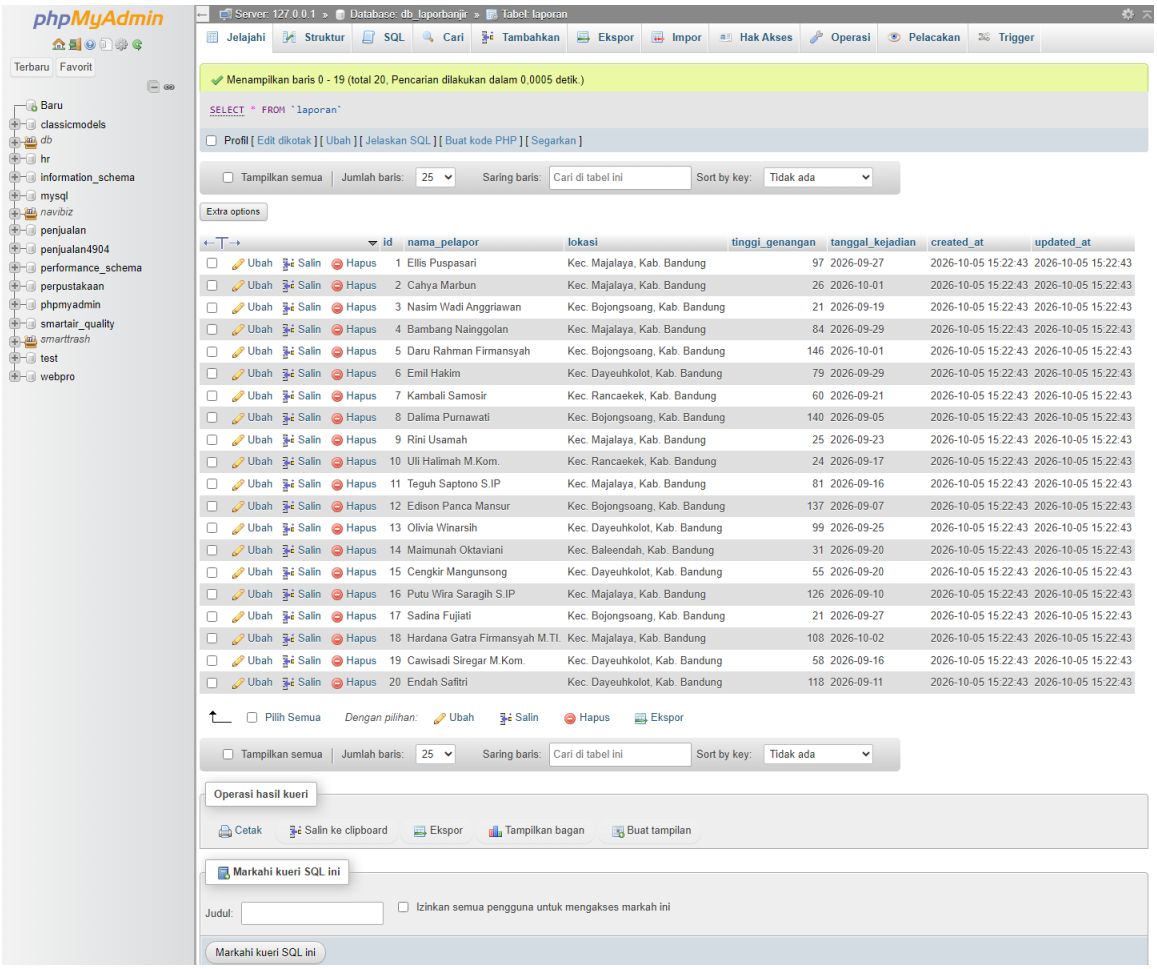
Gambar 3 menampilkan form pelaporan yang dibuka melalui /laporan/create. Form memuat empat input, yaitu nama pelapor, lokasi kejadian, tinggi genangan, dan tanggal kejadian.



Gambar 4 menampilkan halaman konfirmasi. Komponen <x-alert> muncul dengan pesan bahwa laporan berhasil disimpan ke database, diikuti rincian laporan dan status genangannya.



Gambar 5 menampilkan daftar laporan yang dibuka melalui /laporan. Setiap kartu berisi nama pelapor, lokasi, tinggi genangan, tanggal kejadian, dan badge status. Status yang muncul bervariasi, mulai dari Waspada, Siaga, sampai Awas, sesuai tinggi genangan masing-masing laporan.



Gambar 6 menampilkan 20 baris data hasil seeder pada tabel laporan. Nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian terisi otomatis oleh Faker.

5. Analisis dan Pembahasan

1. Perbedaan aplikasi sebelum dan sesudah menggunakan database.

Pada Modul 3, data laporan disimpan dalam bentuk array di dalam controller. Data itu hanya bertahan selama halaman dibuka dan langsung hilang ketika halaman ditutup, sehingga petugas tidak bisa merekap laporan yang pernah masuk. Setelah terhubung ke database, data dikirim dari form dan disimpan ke tabel laporan, lalu dibaca kembali dari tabel tersebut. Perubahan ini membuat data tetap ada walaupun aplikasi ditutup, dan halaman daftar laporan selalu menampilkan data terbaru.

2. Alasan pemilihan struktur tabel dan tipe data.

Tabel laporan memakai kolom id sebagai primary key agar setiap laporan punya penanda unik. Kolom nama_pelapor dan lokasi memakai tipe string karena isinya teks pendek. Kolom tinggi_genangan memakai integer karena nilainya berupa angka bulat dalam satuan sentimeter,

sehingga bisa dibandingkan langsung untuk menentukan status. Kolom tanggal_kejadian memakai tipe date karena hanya menyimpan tanggal tanpa jam, sedangkan created_at dan updated_at diisi otomatis oleh timestamps() untuk mencatat waktu data dibuat dan diubah.

3. Perbedaan peran migration dan seeder.

Migration dan seeder punya tugas yang berbeda. Migration membentuk struktur tabel, yaitu menentukan nama tabel dan kolom beserta tipe datanya. Seeder mengisi tabel dengan data. Karena itu urutannya tidak bisa dibalik, migration harus dijalankan lebih dulu sebelum seeder bisa memasukkan data. Pada praktikum ini migration membuat tabel laporan, sedangkan LaporanSeeder mengisinya dengan 20 data dummy.

4. Kendala dan penyelesaian.

Selama pengerjaan tidak muncul galat yang menghambat, tetapi ada beberapa hal yang perlu disesuaikan agar tidak menimbulkan galat.

Kolom tanggal_kejadian belum ada di form. Tabel laporan mewajibkan kolom tanggal_kejadian, sedangkan form dari Modul 3 hanya mengirim nama pelapor, lokasi, dan tinggi genangan. Kalau form dibiarkan, penyimpanan akan gagal karena kolom tersebut tidak mendapat nilai. Saya menambahkan input tanggal kejadian pada form agar sesuai dengan struktur tabel.

Mass assignment. Laporan::create() hanya mengisi kolom yang terdaftar pada properti $fillable. Karena itu keempat kolom data harus dicantumkan pada model. Kalau tidak, Laravel menolak proses simpan dengan MassAssignmentException.

Konfigurasi .env belum terbaca. Perubahan pada file .env kadang belum ikut terbaca karena Laravel menyimpan cache konfigurasi. Saya menjalankan php artisan config:clear sebelum migration supaya koneksi benar-benar mengarah ke MySQL.

6. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, aplikasi LaporBanjir berhasil dihubungkan ke database MySQL. Data yang dikirim melalui form sekarang benar-benar tersimpan pada tabel laporan dan bisa ditampilkan kembali kapan saja, sehingga masalah kehilangan data pada versi array teratasi. Proses pembuatan tabel menjadi lebih rapi karena memakai migration, sedangkan pengisian data dummy terbantu dengan seeder dan Faker. Dari praktikum ini saya memahami alur kerja Eloquent mulai dari migration, model, penyimpanan lewat Laporan::create(), sampai pengambilan data dengan Laporan::all().

7. Link GitHub

Kode program lengkap praktikum ini dapat diakses pada repositori berikut.

https://github.com/davinaileenskie1-hue/PABW-Semester-3